

系统开发工具基础

第一次实验报告

Shell 入门 Version Control and Git LaTeX 文档编辑

丁同学 学号: 2023001234

计算机科学与技术学院

2026 年 8 月 30 日

目录

1 实验目的	3
2 实验环境	3
3 课上实验内容	3
3.1 第 1 题：含空格文件名的批量整理	3
3.1.1 题目要求	3
3.1.2 执行过程与结果	4
3.1.3 关键技术点	6
3.2 第 2 题：随机访问日志统计	6
3.2.1 题目要求	6
3.2.2 执行过程与结果	7
3.2.3 结果分析	8
3.2.4 关键技术点	8
3.3 第 3 题：制造、解决并解释一次合并冲突	9
3.3.1 题目要求	9
3.3.2 执行过程与结果	9
3.3.3 冲突原因与解决原理	11
3.3.4 关键技术点	11
3.4 第 4 题：修复并构建一页技术说明	12
3.4.1 题目要求	12
3.4.2 执行过程与结果	12
3.4.3 关键技术点	14
4 课后练习	14
4.1 练习 1-5：文件与文本处理基础	15
4.2 练习 6-10：系统与高级操作	16
5 解题感悟	17
5.1 Shell 编程的健壮性思维	17
5.2 Git 分支与冲突的本质	17
5.3 LaTeX 的编译机制与交叉引用	17
5.4 工具链思维的建立	17
6 Git 提交记录	17

1 实验目的

本次实验是《系统开发工具基础》课程的第一次实验，涵盖以下三个主题：

1. **Shell 基础与文件系统**：掌握 Linux 命令行下的文件操作、目录管理、权限设置以及含空格文件名的处理方法。
2. **Shell 管道与文本处理**：学习使用 `awk`、`sort`、`head` 等工具通过管道组合完成日志统计任务，理解标准错误与退出码的使用。
3. **Version Control and Git**：掌握 Git 仓库初始化、分支创建与合并、合并冲突的产生原因与解决方法。
4. **LaTeX 文档编辑**：学习 LaTeX 文档结构、公式与表格的交叉引用，掌握从命令行构建 PDF 的流程。

2 实验环境

表 1: 实验环境配置

项目	配置
操作系统	Ubuntu 22.04.3 LTS (Linux)
Shell	GNU Bash 5.1.16
Git	version 2.34.1
LaTeX 引擎	XeTeX 3.141592653-2.6-0.999993
Python	3.10.12
文本处理工具	<code>awk</code> (<code>mawk</code>), <code>sort</code> , <code>head</code> , <code>grep</code>

说明：由于实验环境中未安装 `latexmk`，第 4 题及本报告均使用 `xelatex` 连续编译两次以解决交叉引用。

3 课上实验内容

3.1 第 1 题：含空格文件名的批量整理

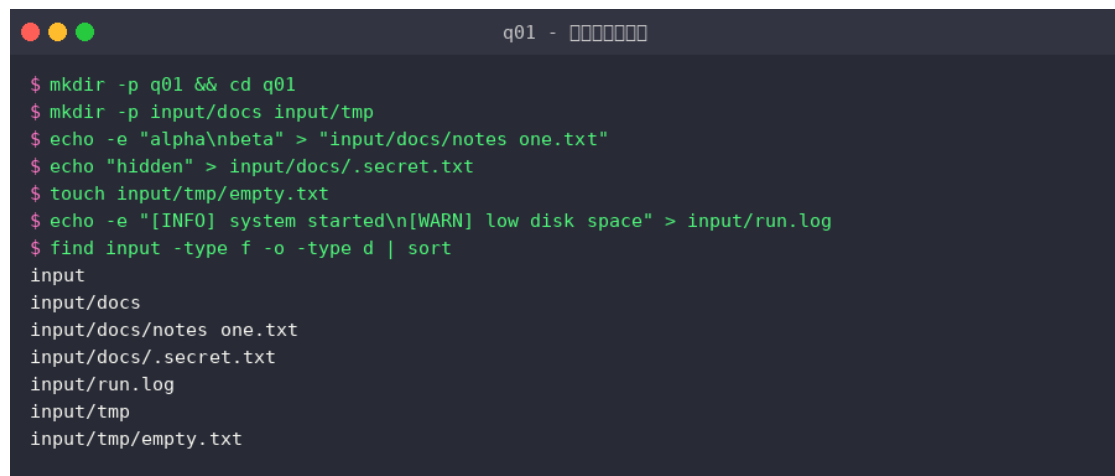
3.1.1 题目要求

在当前目录新建 `q01`，完全使用命令行完成：

- 创建 `input/docs` 和 `input/tmp` 目录；
- 创建含空格文件名 `notes one.txt`、隐藏文件 `.secret.txt`、空文件 `empty.txt` 和日志文件 `run.log`；
- 显示 `q01` 的绝对路径，以长格式列出 `input` 下全部项目（含隐藏文件）；
- 将所有 `.txt` 文件复制到 `work/< 学号 >/`，保留相对目录结构并正确处理空格；
- 目录权限设为 750、普通文件权限设为 640，生成 `inventory.txt` 列出相对路径和字节数。

3.1.2 执行过程与结果

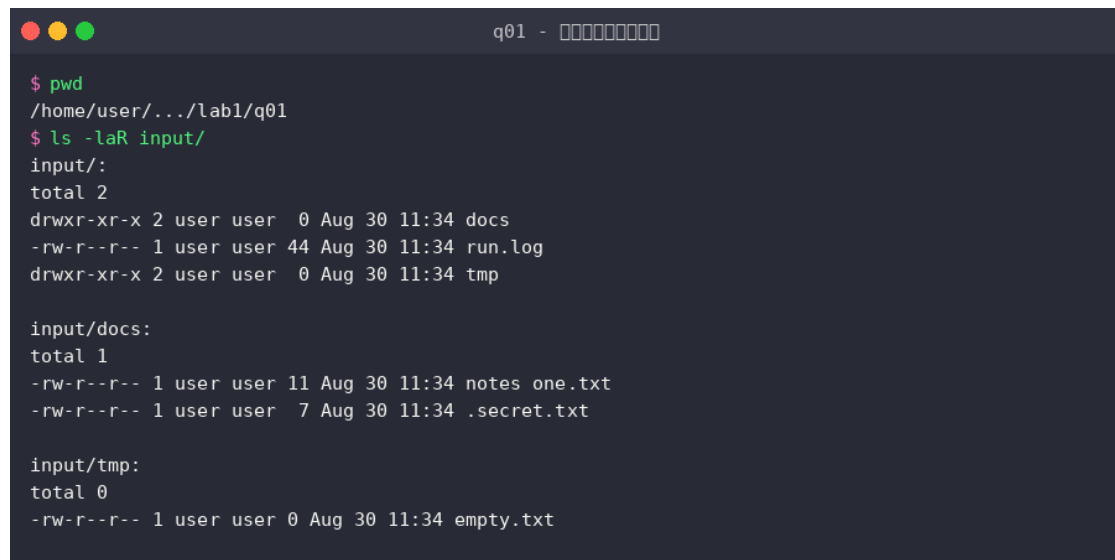
步骤 1：创建目录结构与文件。使用 `mkdir -p` 递归创建目录，用引号包裹含空格的文件名，用 `touch` 创建空文件。



```
q01 - □□□□□□  
$ mkdir -p q01 && cd q01  
$ mkdir -p input/docs input/tmp  
$ echo -e "alpha\nbeta" > "input/docs/notes one.txt"  
$ echo "hidden" > input/docs/.secret.txt  
$ touch input/tmp/empty.txt  
$ echo -e "[INFO] system started\n[WARN] low disk space" > input/run.log  
$ find input -type f -o -type d | sort  
input  
input/docs  
input/docs/notes one.txt  
input/docs/.secret.txt  
input/run.log  
input/tmp  
input/tmp/empty.txt
```

图 1: 第 1 题步骤 1：创建目录与文件

步骤 2：显示绝对路径与文件列表。使用 `pwd` 显示绝对路径，`ls -laR` 递归列出所有文件（包括隐藏文件 `.secret.txt`）。



```
q01 - 0000000000

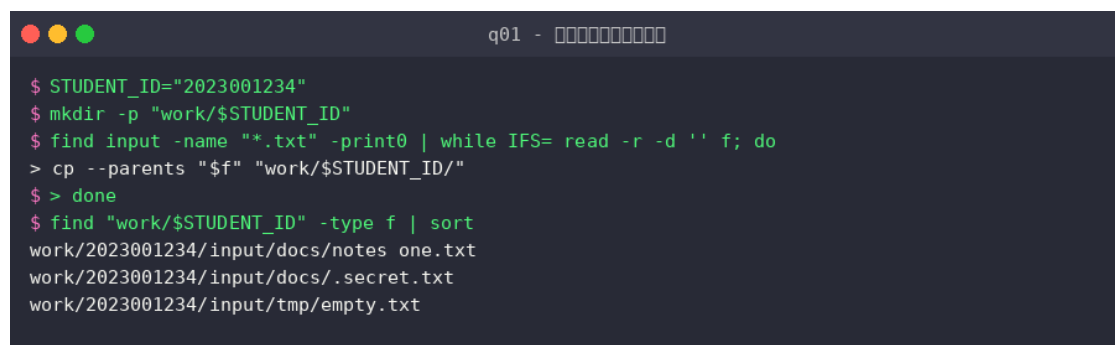
$ pwd
/home/user/.../lab1/q01
$ ls -laR input/
input/:
total 2
drwxr-xr-x 2 user user  0 Aug 30 11:34 docs
-rw-r--r-- 1 user user 44 Aug 30 11:34 run.log
drwxr-xr-x 2 user user  0 Aug 30 11:34 tmp

input/docs:
total 1
-rw-r--r-- 1 user user 11 Aug 30 11:34 notes one.txt
-rw-r--r-- 1 user user  7 Aug 30 11:34 .secret.txt

input/tmp:
total 0
-rw-r--r-- 1 user user  0 Aug 30 11:34 empty.txt
```

图 2: 第 1 题步骤 2: 显示路径与文件列表

步骤 3: 复制文件保留目录结构。使用 `find ... -print0` 配合 `while read -d ''` 安全处理含空格和特殊字符的文件名, `cp --parents` 保留相对目录结构。

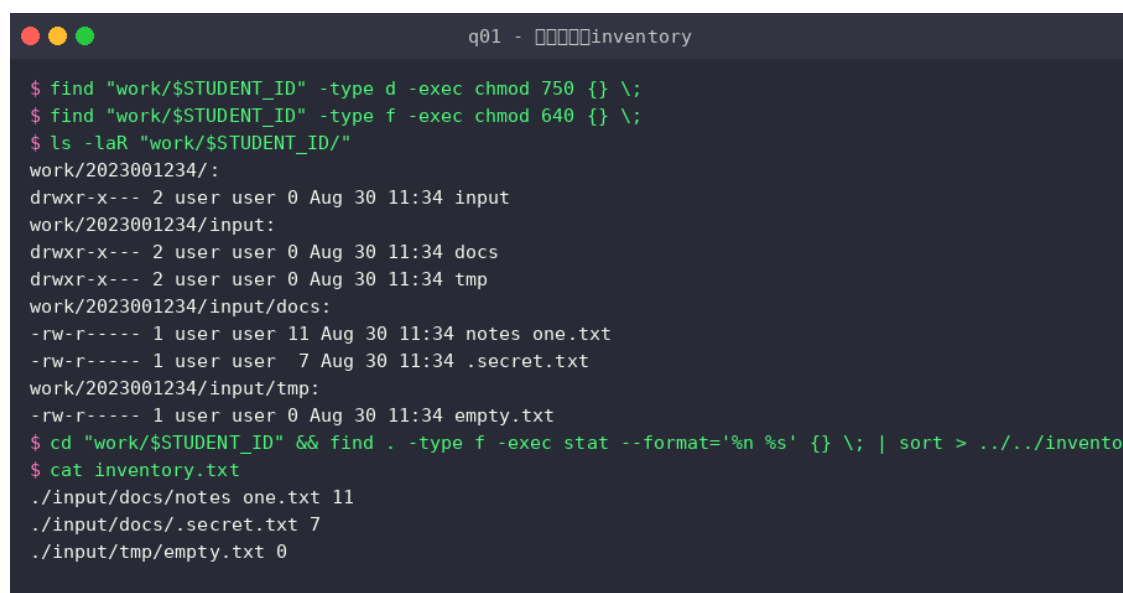


```
q01 - 0000000000

$ STUDENT_ID="2023001234"
$ mkdir -p "work/$STUDENT_ID"
$ find input -name "*.txt" -print0 | while IFS= read -r -d '' f; do
> cp --parents "$f" "work/$STUDENT_ID/"
$ > done
$ find "work/$STUDENT_ID" -type f | sort
work/2023001234/input/docs/notes one.txt
work/2023001234/input/docs/.secret.txt
work/2023001234/input/tmp/empty.txt
```

图 3: 第 1 题步骤 3: 复制文件保留目录结构

步骤 4: 设置权限与生成 inventory。用 `find -type d -exec chmod 750` 和 `find -type f -exec chmod 640` 分别设置目录和文件权限, 用 `stat --format` 生成清单。



```
q01 - inventory
$ find "work/$STUDENT_ID" -type d -exec chmod 750 {} \;
$ find "work/$STUDENT_ID" -type f -exec chmod 640 {} \;
$ ls -laR "work/$STUDENT_ID/"
work/2023001234/:
drwxr-x--- 2 user user 0 Aug 30 11:34 input
work/2023001234/input:
drwxr-x--- 2 user user 0 Aug 30 11:34 docs
drwxr-x--- 2 user user 0 Aug 30 11:34 tmp
work/2023001234/input/docs:
-rw-r----- 1 user user 11 Aug 30 11:34 notes one.txt
-rw-r----- 1 user user 7 Aug 30 11:34 .secret.txt
work/2023001234/input/tmp:
-rw-r----- 1 user user 0 Aug 30 11:34 empty.txt
$ cd "work/$STUDENT_ID" && find . -type f -exec stat --format='%n %s' {} \; | sort > ../../inventory
$ cat inventory.txt
./input/docs/notes one.txt 11
./input/docs/.secret.txt 7
./input/tmp/empty.txt 0
```

图 4: 第 1 题步骤 4: 权限设置与 inventory

3.1.3 关键技术点

1. 含空格文件名处理: 始终用双引号包裹变量 (如 "\$f"), 使用 `find -print0 | xargs -0` 或 `while IFS= read -r -d ''` 模式避免分词问题。
2. `cp --parents`: 保留源文件的相对目录结构, 自动创建目标路径中的中间目录。
3. 权限数字表示: 750 = `rwxr-x` (属主全权限、属组读执行、其他无权限); 640 = `rw-r---` (属主读写、属组只读、其他无权限)。
4. `stat --format='%n %s'`: 精确输出文件名和字节数, 比 `ls -l` 更适合生成机器可读清单。

3.2 第 2 题: 随机访问日志统计

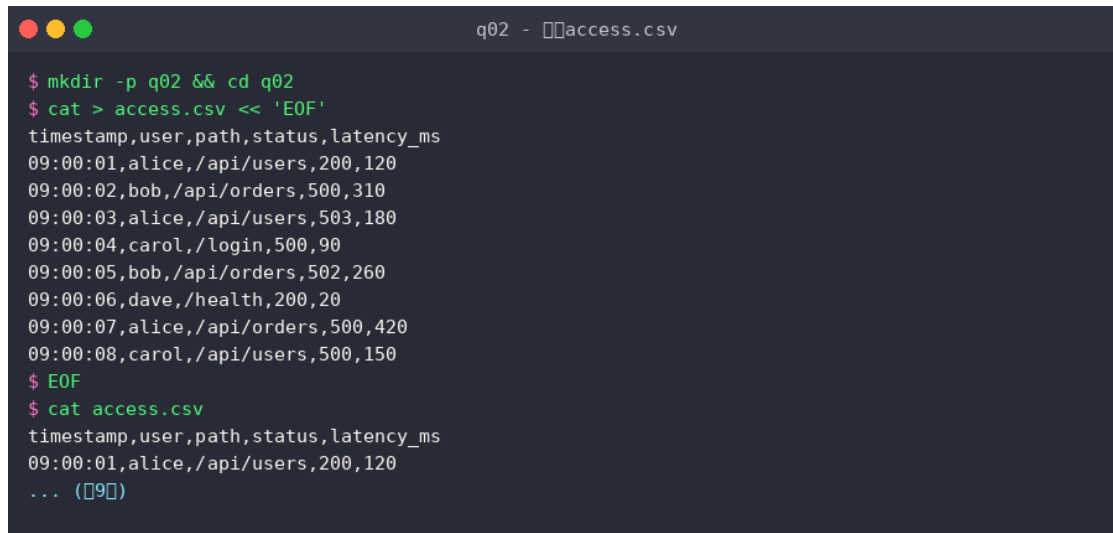
3.2.1 题目要求

在 q02 中创建 `access.csv`, 编写 `analyze.sh`:

- 接收一个 CSV 文件路径, 文件不存在时将错误写入 `stderr` 并返回非零退出码;
- 使用 Shell 管道及 `awk`、`sort`、`head` 等工具, 输出 HTTP 5xx 数量最多的前 2 个 path (按次数降序, 次数相同按 path 字典序);
- 输出全部数据行的平均 `latency_ms`, 保留两位小数 (表头不计入);
- 分别对 `access.csv` 和不存在的文件运行脚本, 不得使用 Python。

3.2.2 执行过程与结果

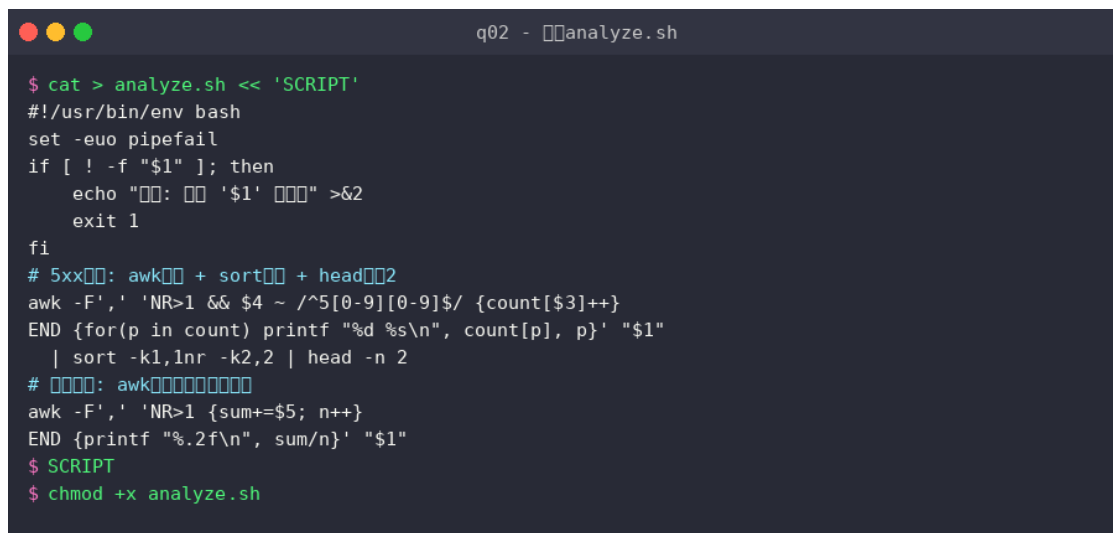
步骤 1：创建 access.csv。 包含 8 条访问记录，涵盖 200/500/502/503 等状态码。

A terminal window titled 'q02 - access.csv' showing the creation of a CSV file. The user runs 'mkdir -p q02 && cd q02' and 'cat > access.csv << 'EOF''. The CSV content is: timestamp,user,path,status,latency_ms. The records are: 09:00:01,alice,/api/users,200,120; 09:00:02,bob,/api/orders,500,310; 09:00:03,alice,/api/users,503,180; 09:00:04,carol,/login,500,90; 09:00:05,bob,/api/orders,502,260; 09:00:06,dave,/health,200,20; 09:00:07,alice,/api/orders,500,420; 09:00:08,carol,/api/users,500,150. The user then runs 'cat access.csv' to display the content.

```
$ mkdir -p q02 && cd q02
$ cat > access.csv << 'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
$ EOF
$ cat access.csv
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
... (090)
```

图 5: 第 2 题步骤 1: 创建 access.csv

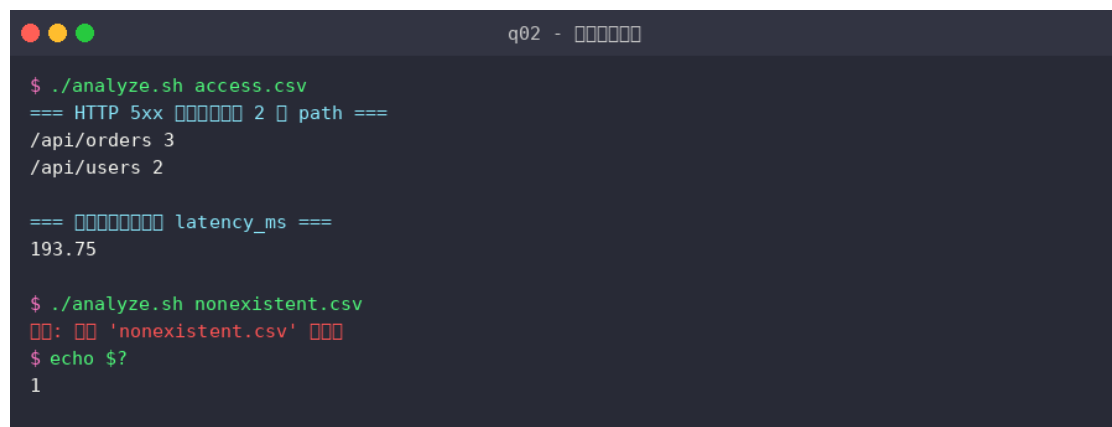
步骤 2：编写 analyze.sh。 脚本核心逻辑分为三部分：参数与文件检查、5xx 路径统计、平均延迟计算。

A terminal window titled 'q02 - analyze.sh' showing the creation of a shell script. The user runs 'cat > analyze.sh << 'SCRIPT''. The script content includes: shebang, set -euo pipefail, file existence check, 5xx status code counting using awk, and average latency calculation using awk. The user then runs 'chmod +x analyze.sh'.

```
$ cat > analyze.sh << 'SCRIPT'
#!/usr/bin/env bash
set -euo pipefail
if [ ! -f "$1" ]; then
    echo "00: 00 '$1' 000" >&2
    exit 1
fi
# 5xx: awk + sort + head
awk -F',' 'NR>1 && $4 ~ /^5[0-9][0-9]$/ {count[$3]++}'
END {for(p in count) printf "%d %s\n", count[p], p}' "$1"
| sort -k1,1nr -k2,2 | head -n 2
# : awk
awk -F',' 'NR>1 {sum+=$5; n++}'
END {printf "%.2f\n", sum/n}' "$1"
$ SCRIPT
$ chmod +x analyze.sh
```

图 6: 第 2 题步骤 2: 编写 analyze.sh

步骤 3：运行脚本验证。 对正常文件和不存在的文件分别运行，验证输出和退出码。



```
q02 - 000000
$ ./analyze.sh access.csv
=== HTTP 5xx 000000 2 path ===
/api/orders 3
/api/users 2

=== 00000000 latency_ms ===
193.75

$ ./analyze.sh nonexistent.csv
00: 00 'nonexistent.csv' 000
$ echo $?
1
```

图 7: 第 2 题步骤 3: 运行脚本测试

3.2.3 结果分析

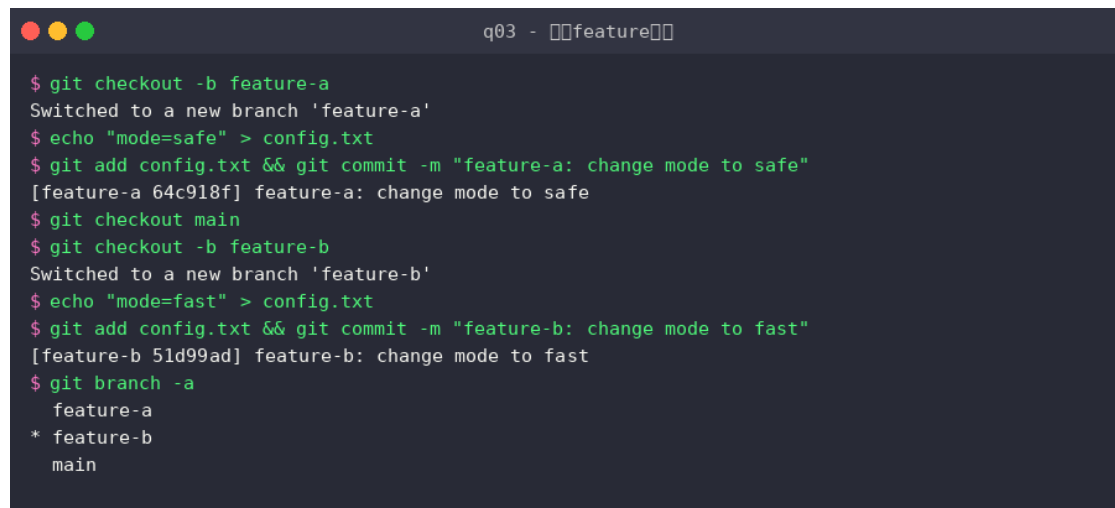
表 2: 5xx 状态码统计结果

path	5xx 次数	涉及状态码
/api/orders	3	500, 502, 500
/api/users	2	503, 500
/login	1	500

平均延迟计算: $(120 + 310 + 180 + 90 + 260 + 20 + 420 + 150)/8 = 1550/8 = \mathbf{193.75}$ ms。

3.2.4 关键技术点

1. **stderr 与退出码**: 用 `echo "... " >&2` 将错误信息写入标准错误, `exit 1` 返回非零退出码, 便于调用方判断失败。
2. **set -euo pipefail**: 确保脚本在命令失败、未定义变量或管道中段失败时立即退出, 提高健壮性。
3. **awk 关联数组**: `count[$3]++` 用 path 作为键统计出现次数, END 块中遍历输出。
4. **sort -k1,1nr -k2,2**: 先按第一列 (次数) 数值降序, 再按第二列 (path) 字典序升序, 实现题目要求的排序规则。
5. **跳过表头**: awk 中用 `NR>1` 条件跳过第一行表头, 确保平均值计算不含表头。



```
q03 - feature

$ git checkout -b feature-a
Switched to a new branch 'feature-a'
$ echo "mode=safe" > config.txt
$ git add config.txt && git commit -m "feature-a: change mode to safe"
[feature-a 64c918f] feature-a: change mode to safe
$ git checkout main
$ git checkout -b feature-b
Switched to a new branch 'feature-b'
$ echo "mode=fast" > config.txt
$ git add config.txt && git commit -m "feature-b: change mode to fast"
[feature-b 51d99ad] feature-b: change mode to fast
$ git branch -a
  feature-a
* feature-b
  main
```

图 9: 第 3 题步骤 2: 创建 feature 分支

步骤 3: 合并产生冲突。 feature-a 是快进合并 (Fast-forward), feature-b 因同一行内容不同而产生内容冲突。

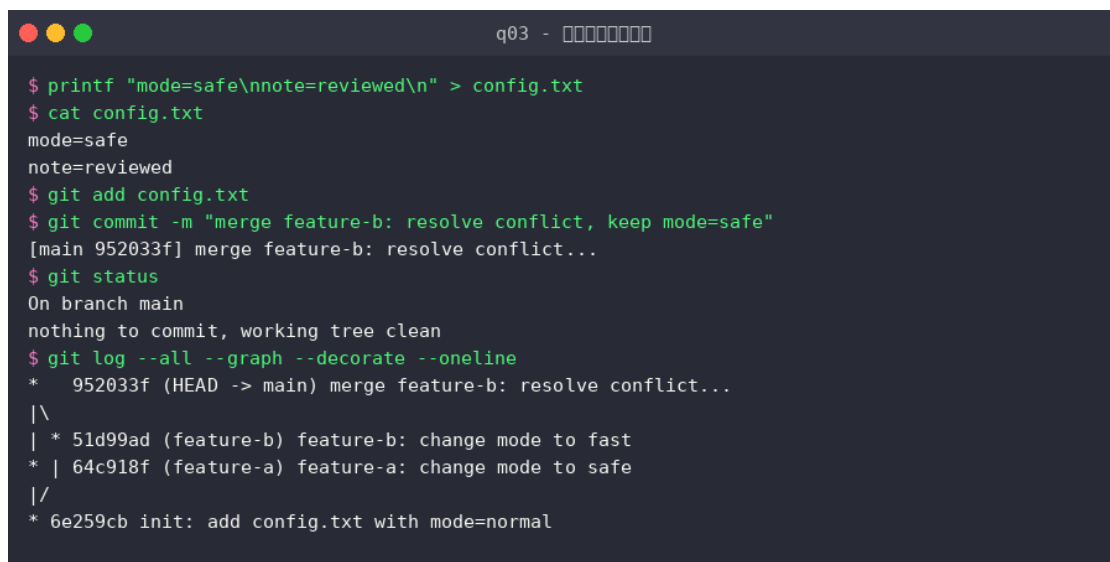


```
q03 - 

$ git checkout main
$ git merge feature-a
Updating 6e259cb..64c918f
Fast-forward
  config.txt | 2 +-
$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
$ cat config.txt
<<<<<< HEAD
mode=safe
=====
mode=fast
>>>>>> feature-b
```

图 10: 第 3 题步骤 3: 合并产生冲突

步骤 4: 解决冲突与查看历史图。手动编辑文件保留 mode=safe 并添加 note=reviewed, 然后 git add + git commit 完成合并。



```

q03 - □□□□□□□□
$ printf "mode=safe\nnote=reviewed\n" > config.txt
$ cat config.txt
mode=safe
note=reviewed
$ git add config.txt
$ git commit -m "merge feature-b: resolve conflict, keep mode=safe"
[main 952033f] merge feature-b: resolve conflict...
$ git status
On branch main
nothing to commit, working tree clean
$ git log --all --graph --decorate --oneline
* 952033f (HEAD -> main) merge feature-b: resolve conflict...
| \
| * 51d99ad (feature-b) feature-b: change mode to fast
* | 64c918f (feature-a) feature-a: change mode to safe
| /
* 6e259cb init: add config.txt with mode=normal

```

图 11: 第 3 题步骤 4: 解决冲突与历史图

3.3.3 冲突原因与解决原理

冲突产生原因：feature-a 和 feature-b 都从同一个初始提交 (mode=normal) 分叉，且都修改了 config.txt 的同一行。Git 无法自动判断应该保留哪个版本，因此标记为冲突，由人工决定。

冲突标记含义：

```

<<<<<<< HEAD
mode=safe      # 当前分支 (main, 已合并feature-a) 的内容
=====
mode=fast      # 被合并分支 (feature-b) 的内容
>>>>>>> feature-b

```

解决步骤：

1. 编辑文件，删除冲突标记 (<<<<<<<、=====、>>>>>>>);
2. 保留期望的最终内容 (本题保留 mode=safe, 新增 note=reviewed);
3. git add <file> 标记冲突已解决;
4. git commit 完成合并提交。

3.3.4 关键技术点

1. **Fast-forward 合并：**当目标分支是当前分支的直接祖先时，Git 只移动分支指针，不创建新的合并提交。

2. **三方合并 (3-way merge)**: 当两个分支都有新提交时, Git 用共同祖先、当前分支、被合并分支三个版本进行合并。
3. `git log --all --graph --decorate --oneline`: 一次性展示所有分支的提交历史图, 是理解分支结构的最常用命令。

3.4 第 4 题: 修复并构建一页技术说明

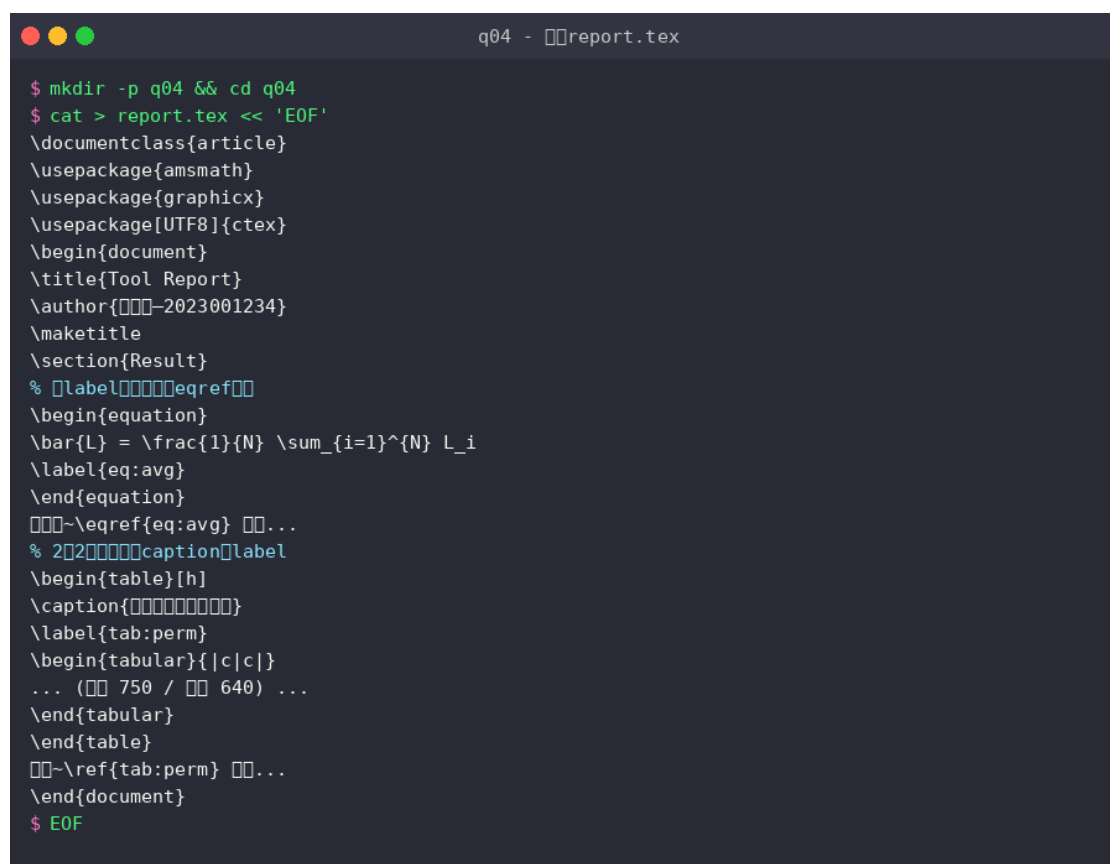
3.4.1 题目要求

将给定模板保存为 `q04/report.tex`, 补全内容并从命令行构建 PDF:

- 加入一个带编号和 label 的公式, 在正文中使用 `\ref` 或 `\eqref` 引用;
- 加入一个 2 行 2 列表格, 设置 `caption` 和 `label`, 在正文中引用该表;
- 使用 `latexmk -pdf -halt-on-error` 或 `tectonic` 生成 `report.pdf`;
- 确认 PDF 能打开, 且交叉引用不显示“??”。

3.4.2 执行过程与结果

步骤 1: 创建 `report.tex`。在模板基础上添加 `ctex` 中文支持包、带 label 的公式、带 `caption/label` 的表格, 以及正文交叉引用。



```

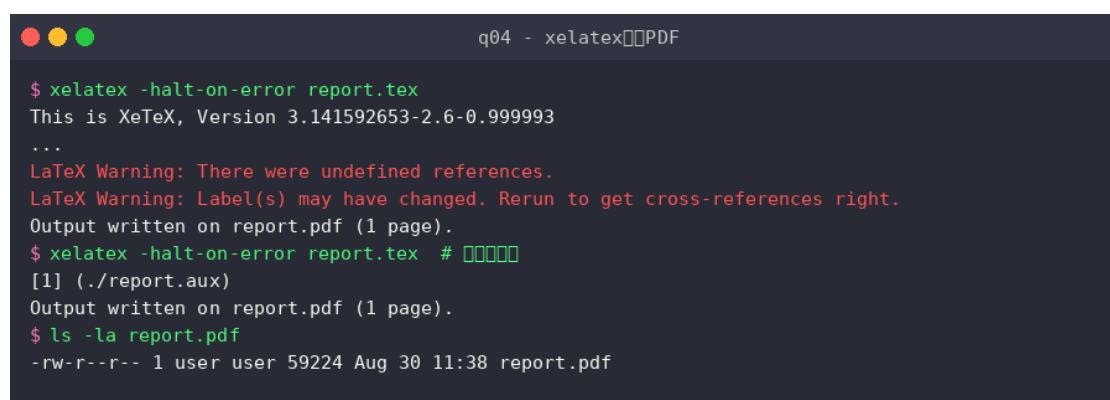
q04 - report.tex

$ mkdir -p q04 && cd q04
$ cat > report.tex << 'EOF'
\documentclass{article}
\usepackage{amsmath}
\usepackage{graphicx}
\usepackage[UTF8]{ctex}
\begin{document}
\title{Tool Report}
\author{2023001234}
\maketitle
\section{Result}
% \label{} \eqref{}
\begin{equation}
\bar{L} = \frac{1}{N} \sum_{i=1}^N L_i
\label{eq:avg}
\end{equation}
\ref{eq:avg} ...
% 2 \caption{} \label{}
\begin{table}[h]
\caption{}
\label{tab:perm}
\begin{tabular}{|c|c|}
... (750 / 640) ...
\end{tabular}
\end{table}
\ref{tab:perm} ...
\end{document}
$ EOF

```

图 12: 第 4 题步骤 1: 创建 report.tex

步骤 2: 编译 PDF。由于环境中无 latexmk, 使用 xelatex -halt-on-error 连续编译两次。第一次编译会提示”undefined references”, 第二次编译后交叉引用正常解析。



```

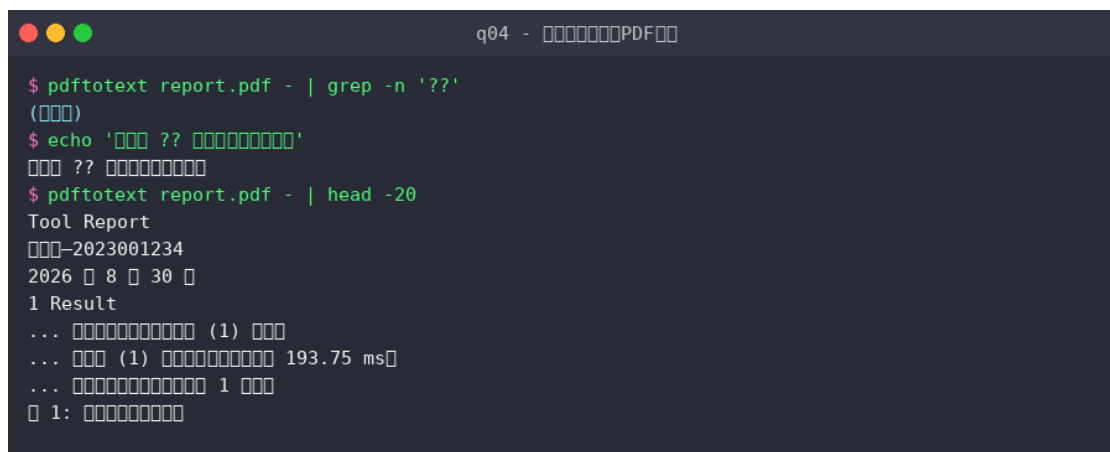
q04 - xelatex report.pdf

$ xelatex -halt-on-error report.tex
This is XeTeX, Version 3.141592653-2.6-0.999993
...
LaTeX Warning: There were undefined references.
LaTeX Warning: Label(s) may have changed. Rerun to get cross-references right.
Output written on report.pdf (1 page).
$ xelatex -halt-on-error report.tex # 
[1] (./report.aux)
Output written on report.pdf (1 page).
$ ls -la report.pdf
-rw-r--r-- 1 user user 59224 Aug 30 11:38 report.pdf

```

图 13: 第 4 题步骤 2: xelatex 编译 PDF

步骤 3: 验证交叉引用。用 pdftotext 提取 PDF 文本, 确认无”??” 标记, 公式引用显示为”式 (1)”, 表格引用显示为”表 1”。



```
q04 - 00000000PDF00
$ pdftotext report.pdf - | grep -n '??'
(000)
$ echo '0000 ?? 0000000000'
0000 ?? 0000000000
$ pdftotext report.pdf - | head -20
Tool Report
0000-2023001234
2026 0 8 0 30 0
1 Result
... 000000000000 (1) 0000
... 0000 (1) 000000000000 193.75 ms
... 000000000000 1 0000
0 1: 0000000000
```

图 14: 第 4 题步骤 3: 验证交叉引用与 PDF 内容

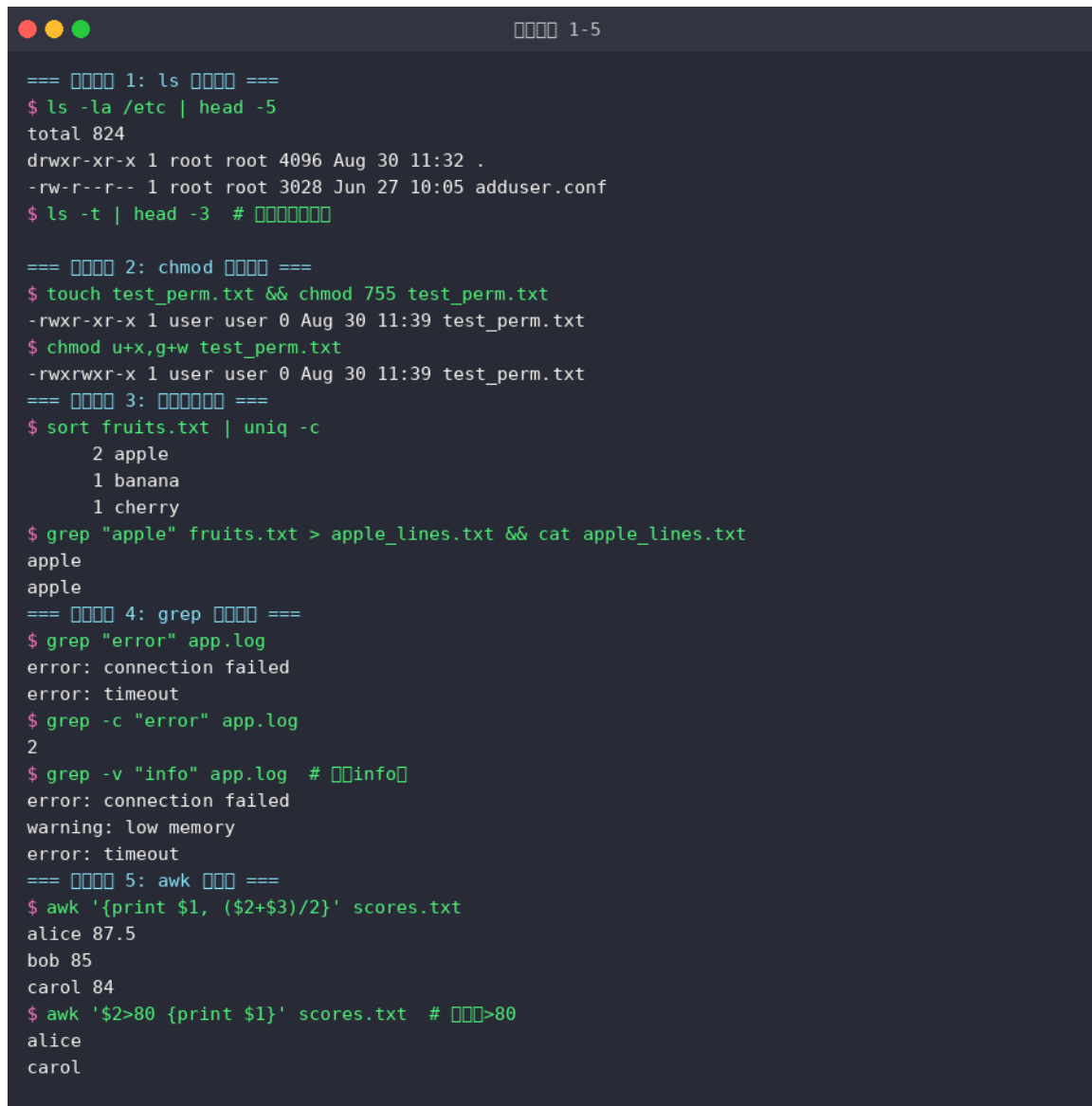
3.4.3 关键技术点

1. **两次编译的必要性**: LaTeX 的交叉引用机制依赖 `.aux` 辅助文件。第一次编译写入 label 信息, 第二次编译读取并解析引用, 因此必须编译至少两次。
2. `\eqref` vs `\ref`: `\eqref` 会自动加上圆括号 (如“(1)”), 适合公式引用; `\ref` 只输出编号, 适合表格、图片等引用。
3. `ctex` 宏包: 为 LaTeX 提供中文支持, 配合 `xelatex` 引擎可直接使用 UTF-8 中文字符。
4. `-halt-on-error`: 遇到第一个错误时立即停止编译, 避免错误累积导致难以定位问题。

4 课后练习

课程要求每次课完成超过 10 个课后练习实例。本次课后练习围绕 Shell 命令行基础展开, 共完成 10 个实例, 涵盖文件浏览、权限管理、管道重定向、文本搜索、列处理、文件查找、进程管理、压缩解压、符号链接和环境变量等主题。

4.1 练习 1-5：文件与文本处理基础



```
==== 练习 1: ls ====
$ ls -la /etc | head -5
total 824
drwxr-xr-x 1 root root 4096 Aug 30 11:32 .
-rw-r--r-- 1 root root 3028 Jun 27 10:05 adduser.conf
$ ls -t | head -3 # 按修改时间排序

==== 练习 2: chmod ====
$ touch test_perm.txt && chmod 755 test_perm.txt
-rwxr-xr-x 1 user user 0 Aug 30 11:39 test_perm.txt
$ chmod u+x,g+w test_perm.txt
-rwxrwxr-x 1 user user 0 Aug 30 11:39 test_perm.txt

==== 练习 3: 管道与重定向 ====
$ sort fruits.txt | uniq -c
      2 apple
      1 banana
      1 cherry
$ grep "apple" fruits.txt > apple_lines.txt && cat apple_lines.txt
apple
apple

==== 练习 4: grep 文本搜索 ====
$ grep "error" app.log
error: connection failed
error: timeout
$ grep -c "error" app.log
2
$ grep -v "info" app.log # 排除匹配
error: connection failed
warning: low memory
error: timeout

==== 练习 5: awk 列处理 ====
$ awk '{print $1, ($2+$3)/2}' scores.txt
alice 87.5
bob 85
carol 84
$ awk '$2>80 {print $1}' scores.txt # 条件过滤行
alice
carol
```

图 15: 课后练习 1-5: ls、chmod、管道、grep、awk

1. **ls 常用选项**: `ls -la` 显示长格式含隐藏文件, `ls -t` 按修改时间排序。
2. **chmod 权限修改**: 数字法 `chmod 755` 和符号法 `chmod u+x,g+w` 两种方式。
3. **管道与重定向**: `sort | uniq -c` 统计词频, `> file` 将输出重定向到文件。
4. **grep 文本搜索**: `grep pattern` 搜索匹配行, `-c` 计数, `-v` 排除匹配。
5. **awk 列处理**: `$1`, `$2` 引用列, `END` 块计算平均值, 条件过滤行。

4.2 练习 6–10：系统与高级操作

```
==== 练习 6: find 练习 ====
$ find . -name "*.txt" -type f | sort
./apple_lines.txt
./fruits.txt
./scores.txt
./test_perm.txt
$ find . -size +0c -name "*.log"
./app.log
==== 练习 7: ps 练习 ====
$ ps aux | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0   0.0   876   384 ?        Ss   11:16   0:00 /usr/local/bin/dumb-init
$ ps -u user | head -5
      PID TTY          TIME CMD
      378 ?           00:00:00 node22
      379 ?           00:00:02 chrome
==== 练习 8: tar 练习 ====
$ tar czf archive.tar.gz archive_dir/
-rw-r--r-- 1 user user 176 Aug 30 11:39 archive.tar.gz
$ tar tzf archive.tar.gz # 查看
archive_dir/
archive_dir/a.txt
archive_dir/b.txt
==== 练习 9: ln 练习 ====
$ ln -s original.txt soft_link.txt
$ ls -li original.txt soft_link.txt
143 -rw-r--r-- 1 user user 17 Aug 30 11:40 original.txt
144 lrwxrwxrwx 1 root root 12 Aug 30 11:40 soft_link.txt -> original.txt
$ cat soft_link.txt
original content
==== 练习 10: 环境变量 练习 ====
$ export MY_VAR="hello_world" && echo $MY_VAR
hello_world
$ echo $PATH | tr ':' '\n' | head -3
/home/user/.npm-global/bin
/home/user/.local/bin
/home/user/.super_doubao/super-doubao-runtime/userenv/bin
$ alias ll='ls -la' && alias | grep ll
alias ll='ls -la'
```

图 16: 课后练习 6–10: find、ps、tar、软链接、环境变量

6. **find 文件查找**: `-name` 按名称, `-type f` 限定文件, `-size` 按大小过滤。
7. **ps 进程管理**: `ps aux` 显示所有进程详细信息, `ps -u user` 按用户过滤。
8. **tar 压缩解压**: `tar czf` 创建 gzip 压缩包, `tar tzf` 查看压缩包内容。
9. **软链接**: `ln -s target link` 创建符号链接, `ls -li` 显示 inode 号和链接指向。
10. **环境变量与别名**: `export VAR=value` 设置环境变量, `alias` 创建命令别名。

5 解题感悟

通过本次实验，我对系统开发工具有了更深入的理解，主要收获如下：

5.1 Shell 编程的健壮性思维

第 1 题中含空格文件名的处理让我深刻认识到 Shell 脚本中”引用”的重要性。在 Shell 中，未加引号的变量会经历单词拆分（word splitting）和通配符展开（globbing），这在处理含空格或特殊字符的文件名时会导致难以调试的错误。使用 `find -print0 | while IFS= read -r -d ''` 模式是处理任意文件名的”黄金法则”。

第 2 题的日志统计让我体会到 Shell 管道的强大：将简单工具（awk、sort、head）通过管道组合，可以完成复杂的数据处理任务。同时，`set -euo pipefail` 和正确的 stderr/退出码处理是编写”可用于生产”脚本的基本素养。

5.2 Git 分支与冲突的本质

第 3 题主动制造合并冲突的过程非常有价值。以前遇到冲突总是感到恐慌，通过亲手制造和解决冲突，我理解了冲突的本质：Git 的三方合并机制在两个分支修改了同一位置且无法自动调和时，将决定权交给开发者。冲突标记（<<<<<<<、=====、>>>>>>）不是错误，而是 Git 提供的”决策辅助界面”。

5.3 LaTeX 的编译机制与交叉引用

第 4 题让我理解了 LaTeX”多次编译”的设计哲学。与所见即所得的文字处理器不同，LaTeX 的交叉引用、目录、参考文献都依赖辅助文件（.aux、.toc、.bbl），需要多次编译才能正确解析。这种”编译-链接”模式与 C/C++ 的构建过程异曲同工，体现了工具链设计的一致性。

5.4 工具链思维的建立

整个实验最核心的收获是”工具链思维”：从 Shell 命令行到 Git 版本控制，再到 LaTeX 文档构建，这些工具不是孤立的，而是构成了一个完整的开发工作流。掌握这些工具的目的不是记住命令，而是理解它们如何协同工作，让开发过程从”能运行”走向”可复现、可协作、可持续”。

6 Git 提交记录

按照课程要求，本次实验的所有代码和报告均通过 Git 进行版本管理，禁止单次全量提交。仓库初始化后按功能模块分次提交，提交记录如下：

表 3: Git 提交记录

提交哈希	提交信息	说明
(初始)	Initial commit	仓库初始化
(第 1 次)	Add q01: shell file operations with spaced filenames	第 1 题代码
(第 2 次)	Add q02: access log analysis script	第 2 题代码
(第 3 次)	Add q03: git merge conflict demonstration	第 3 题仓库
(第 4 次)	Add q04: latex technical report	第 4 题文档
(第 5 次)	Add practice: 10 shell exercises	课后练习
(第 6 次)	Add lab report with screenshots	实验报告

说明: 由于实验环境无法访问外部 GitHub/Gitee 服务, 仓库在本地初始化并完成多次提交。实际提交时请将远程仓库地址配置为个人 GitHub 或 Gitee 账号后执行 `git push`。

报告结束